PENSE-BÊTE · RECOMMANDATIONS DE SÉCURITÉ

Sécurisation du protocole OpenID Connect

03-400 ANSSI - Sécurisation de la mise en œuvre du protocole OpenID Connect — source : fiche de révision du module (gemma4, 03/11/2026 ; relu par Claude le 04/11/2026)

À RETENIR

1. Le protocole OIDC repose sur OAuth2.0 en ajoutant l'ID Token pour récupérer l'identité d'un utilisateur authentifié.
2. La cinématique authorization code est recommandée en priorité pour les clients confidentiels et les applications Web.
3. L'algorithme de signature doit être fixé hors bande lors de l'enrôlement pour éviter l'utilisation du champ alg.

Présentation du protocole OIDC

Notion	À retenir
Acteurs et échanges	Acteurs : **utilisateur**, **fournisseur de service** (client **confidentiel** ou **public**), **fournisseur d'identité**, **fournisseur de données**. Échanges : `claims`, `authorization code`, `access token` et `ID Token` (**JWS**).
Choix cinématique	Cinématique `authorization code` : recommandée en priorité pour les **clients confidentiels** et **applications Web**. Protège la confidentialité de l'**access token** et permet l'authentification via le **Token Endpoint**.

Contrôle de la sécurisation des communications

- ☐ Les communications HTTP utilisent-elles **TLS** (selon le guide de l'ANSSI) ?
- ☐ Le **nom de domaine** du fournisseur d'identité correspond-il au **SubjectAltName** du certificat ?
- ☐ Le **certificate pinning** est-il mis en œuvre (certificat, empreinte ou clé publique) ?

Sécurisation de la demande d'authentification

Notion	À retenir
Intégrité paramètres	Envoyer les paramètres dans un **JWS** protégé par un **HMAC** ou une **signature électronique**. Un seul mode autorisé par client, fixé à l'**enrôlement**.
CSRF, rejeu, redirections	Paramètre `state` (**128 bits** / **22 caractères ASCII**) en **cookie de session** contre **CSRF** ; `nonce` aléatoire contre **rejeu** ; **liste blanche** d'URL de redirection contre **redirections ouvertes**.

Sécurisation des jetons et de la réponse

Notion	À retenir
L'authorization code	Généré avec **128 bits** d'entropie, durée de vie de **quelques minutes**, associé à l'**identifiant du client**, stocké en **SHA256**, utilisé **une seule fois**.
L'access token et l'ID Token	Access token : **128 bits**, haché **SHA256** en base, **uniquement** via l'entête **Authorization**, jamais journalisé, validité courte. ID Token : vérifier **nonce**, **iss**, **aud** (et **acr** si niveau exigé) ; HMAC avec un secret ≠ `client_secret`.
La récupération du UserInfo	Access token transmis par l'entête **Authorization**. Croiser l'identifiant **sub** du UserInfo avec celui de l'**ID Token** pour détecter une incohérence.

Gestion des secrets et des signatures

Notion	À retenir
Secrets partagés	Générés avec **entropie minimale de 128 bits**, renouvelés régulièrement, **différents par client** et accès strictement restreint.
JSON Web Signature	Utiliser **certificats d'autorité de confiance** avec vérification de révocation. Algorithme fixé à l'enrôlement (**SHA256 minimum**). Ne pas utiliser le champ `alg`.
Le champ `alg`	Ne pas l'utiliser pour déterminer l'algorithme car il permet de forger des JWS (ex: valeur `none` désactivant l'intégrité).

Sécurisation applicative et enrôlement

Notion	À retenir
Sécurité Web et journalisation	Suivre le guide **ANSSI** : validation **stricte** des paramètres, requêtes **SQL paramétrées**, durcissement et architecture **n-tiers**. Journaliser les événements d'**authentification**.
Enrôlement des clients OIDC	Ne pas combiner **découverte automatisée** et **enrôlement automatisé** pour éviter l'attaque **IdP Mix-Up**. Vérifier l'identité des fournisseurs de service selon la **sensibilité** des informations.

Pièges et confusions

- ⚠ Confondre l'utilité du paramètre state (contre CSRF) avec celle du paramètre nonce (contre le rejeu).
- ⚠ Utiliser le champ alg d'un JWS pour déterminer l'algorithme de vérification au lieu de le fixer à l'enrôlement.
- ⚠ Croire que la valeur none dans le champ alg garantit l'intégrité alors qu'elle la supprime.
- ⚠ Oublier que la capture d'un access token équivaut à une usurpation complète des droits jusqu'à son expiration.
- ⚠ Ne pas vérifier l'URL de redirection, ce qui expose au risque de redirection ouverte et au vol de l'authorization code.

Sigles et définitions

OIDC	Protocole reposant sur OAuth2.0 permettant de récupérer l'identité d'un utilisateur authentifié.
OAuth2.0	Cadre définissant des mécanismes pour obtenir l'autorisation d'accéder à des ressources protégées.
Client confidentiel	Client opéré dans un environnement maîtrisé permettant de protéger ses secrets d'authentification.
JWT	Standard d'échange d'informations au format JSON, protégé en intégrité par un JWS ou en confidentialité par un JWE.
JWS	JSON Web Signature protégeant l'intégrité d'un JWT.
JWE	JSON Web Encryption assurant la confidentialité d'un JWT.

Auto-test

1. Quel paramètre aléatoire permet de contrer les attaques CSRF ?
2. Quelle est la durée de vie de l'authorization code ?
3. Qu'est-ce qui distingue un client public d'un client confidentiel ?
4. Quels éléments doit-on vérifier dans l'ID Token ?
5. Comment détecter une incohérence lors de la récupération du UserInfo ?

Réponses : **1.** Le paramètre state. · **2.** Quelques minutes. · **3.** Il est opéré sur un environnement non maîtrisé (ex : mobile) et ne peut protéger ses secrets. · **4.** Le nonce, iss, aud et acr. · **5.** En croisant l'identifiant sub du UserInfo avec celui de l'ID Token.

À compléter par la fiche, le podcast et le quiz du module. Rédigé à partir de la fiche de révision ; seuils et chiffres à la date de la source.